

Network Troubleshooting

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

A systematic way to isolate network problems fast – the OSI-layer approach, the core toolset (ping, traceroute, DNS lookups, Wireshark), and a full worked diagnosis from symptom to root cause.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 – 2	Why method beats guesswork, and the OSI-layer troubleshooting approach
3 – 5	ping/traceroute, DNS lookup tools, and reading a Wireshark capture
6 – 7	A full worked diagnosis, and a symptom-to-cause quick reference
8 – 9	Documentation & escalation, and a glossary

How to use this guide: read start to finish for the full picture, or jump to section 6 for the worked "website is slow" walkthrough. Every diagram is numbered and referenced directly in the surrounding text.

Table of Contents

1. Why Method Matters More Than Tools	3
2. The OSI-Layer Troubleshooting Approach	4
3. ping & traceroute	5
4. DNS Troubleshooting: nslookup & dig	6
5. Wireshark: Reading a Packet Capture	7
6. Worked Example: Diagnosing "The Website Is Slow"	8
7. Symptom → Likely Cause Reference	9
8. Documentation & Escalation	10
9. Quick Reference & Glossary	11

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

Why Method Matters More Than Tools

1.1 THE TRAP OF RANDOM GUESSING

Faced with "the network is broken," it's tempting to start randomly trying things – restart the router, flush DNS, check the cable – hoping something works. This sometimes fixes the immediate problem by accident, but teaches nothing about *why* it happened, and wastes enormous time when the actual cause isn't one of the things randomly tried. A **systematic method** instead narrows down the actual point of failure methodically, every single time, regardless of how unfamiliar the specific problem is.

The goal isn't just "fix it" – it's "know why it broke." A restart that fixes a symptom without identifying the root cause means the same failure can, and often will, recur. Systematic troubleshooting trades a little more time up front for actually understanding what failed, which is what prevents repeat incidents.

1.2 THE CORE DISCIPLINE: ISOLATE, DON'T GUESS

Every troubleshooting method covered in this guide shares one underlying principle: systematically **eliminate possibilities** by testing at specific, deliberate points, rather than jumping straight to a suspected fix. Each test should meaningfully narrow the set of things that could still be wrong – that's what separates methodical troubleshooting from trial and error that happens to be organized.

1.3 TOOLS COME SECOND

The rest of this guide covers specific tools – ping, traceroute, DNS lookups, Wireshark – but every one of them is only useful in service of a method. A powerful tool used without a plan just produces more data to be confused by; a clear method with basic tools reliably finds the answer. Section 2's OSI-layer approach is the foundational method everything else in this guide builds on.

The OSI-Layer Troubleshooting Approach

2.1 WORKING THROUGH THE STACK

The OSI model (covered fully in the main Computer Networks guide) doubles as an extremely effective troubleshooting framework: test each layer, from the bottom up, confirming it's working before assuming a problem lives higher in the stack. A "the website won't load" complaint could be caused by anything from an unplugged cable (Layer 1) to a broken application server (Layer 7) – testing layer by layer quickly identifies which one is actually the problem, instead of guessing at the application layer first just because that's where the symptom is visible.

1 · Physical – is it plugged in, powered, link light on?

2 · Data Link – correct VLAN, switch port up?

3 · Network – can you ping the gateway/destination?

4 · Transport – is the right port open/reachable?

5-6 · Session/Presentation – TLS handshake succeeding?

7 · Application – is the actual service responding correctly?

Start at the bottom – don't skip straight to the layer where the symptom appears

Fig 2.1 – Working bottom-up through the OSI layers, confirming each is functional before assuming the problem is higher in the stack.

2.2 WHY BOTTOM-UP (USUALLY)

Lower-layer problems are both more common in practice and produce misleading symptoms at higher layers – a flaky Layer 1 cable can look exactly like an "application is slow" problem if you only look at the application. Starting low and confirming each layer works, in order, avoids chasing a phantom application bug that's actually a bad cable three layers down.

2.3 WHEN TO DEVIATE

Bottom-up isn't a rigid law – if strong evidence points directly at one layer (a specific error message, a known recent change, a symptom that only makes sense at one layer), it's reasonable to test that layer first and work outward from there. The framework's real value is having a complete, ordered checklist to fall back on whenever intuition runs out.

ping & traceroute

3.1 PING: IS IT REACHABLE AT ALL?

ping sends ICMP Echo Request messages to a destination and waits for Echo Replies, testing basic Layer 3 reachability and measuring round-trip latency – exactly the diagnostic tool referenced in the main Computer Networks guide's glossary. It answers a simple but essential question first: can a packet get from here to there and back, at all, before investigating anything more specific.

```
$ ping 8.8.8.8
$ 64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=14.2 ms
$ 64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=13.8 ms
```

3.2 TRACEROUTE: WHERE DOES IT BREAK?

If ping fails, or succeeds with high or inconsistent latency, **traceroute** reveals the actual path a packet takes, hop by hop, and the latency at each one – pinpointing *where* along the path a problem lives, rather than just confirming that one exists. It works by sending packets with increasing TTL values (1, 2, 3...) and capturing the "TTL exceeded" ICMP message each router along the way sends back as its TTL hits zero – each such message reveals one more hop's identity.

```
$ traceroute 8.8.8.8
$ 1 10.0.0.1 2.1 ms
$ 2 203.0.113.1 8.4 ms
$ 3 * * * (timeout – possible problem here)
$ 4 8.8.8.8 14.2 ms
```

A single timeout mid-path isn't always a real problem. Some routers deliberately deprioritize or rate-limit ICMP responses used by traceroute for security or load reasons, producing a timeout at that hop even though traffic is actually flowing through it fine. The signal that matters is a pattern – consistent failure at and beyond a specific hop, or a sudden, sustained latency jump – not a single isolated timeout.

3.3 READING LATENCY JUMPS

A sharp, sustained latency increase that starts at a specific hop and persists through every hop after it usually indicates a real problem at or near that hop – a congested link, an overloaded router, or a long-haul path segment (like an undersea cable) genuinely adding that latency. This is often the single most useful signal traceroute provides.

DNS Troubleshooting: nslookup & dig

4.1 WHY DNS GETS CHECKED EARLY

A huge fraction of "the internet is down" complaints are actually DNS failures – the underlying network connectivity is fine, but the name (like a website's domain) simply isn't resolving to an IP address. Checking DNS resolution early, before assuming a deeper connectivity problem, saves considerable time given how common this specific failure mode is in practice.

4.2 NSLOOKUP: QUICK AND SIMPLE

```
$ nslookup example.com
$ Server: 8.8.8.8
$ Address: 93.184.216.34
```

Confirms whether a name resolves at all, and to what address – the fastest first check for a suspected DNS issue.

4.3 DIG: MORE DETAIL, MORE CONTROL

```
$ dig example.com +short
$ 93.184.216.34
$ dig example.com @1.1.1.1
```

dig exposes far more detail – the specific DNS server queried, response times, the full record set, and TTL values – and lets you query a *specific* DNS server directly (like `@1.1.1.1` above), which is invaluable for isolating whether a problem is with one particular resolver versus DNS resolution generally.

4.4 THE CLASSIC ISOLATION TEST

Comparing results from your normal (often ISP-provided) DNS resolver against a known-good public resolver like `8.8.8.8` or `1.1.1.1` quickly reveals whether a DNS problem is specific to your configured resolver (switch resolvers and it works normally) or genuinely upstream – the domain itself has an actual problem, or its authoritative servers are unreachable from your network entirely.

Wireshark: Reading a Packet Capture

5.1 WHEN PING/TRACEROUTE/DNS TOOLS AREN'T ENOUGH

Sections 3 and 4's tools confirm reachability and name resolution, but sometimes the actual problem lives inside the traffic itself – a malformed request, an unexpected TCP reset, an application timing out for reasons that aren't visible from the outside. **Wireshark** captures and displays every packet crossing an interface, letting you inspect the actual conversation at every layer, from raw Ethernet frames up through full application-layer content.

5.2 THE THREE-PANE VIEW

PANE	SHOWS
Packet List	One row per captured packet – time, source, destination, protocol, brief info
Packet Details	The selected packet broken down layer by layer, matching the OSI structure from Section 2
Packet Bytes	The raw hex/ASCII bytes of the packet, for the deepest level of inspection

5.3 WHAT TO ACTUALLY LOOK FOR

- **TCP resets (RST)** – a connection being abruptly terminated, often indicating a firewall block, a crashed service, or a mismatched expectation between client and server.
- **Retransmissions** – the same segment sent multiple times, a strong signal of packet loss or an unreliable link somewhere along the path.
- **The three-way handshake (or its absence)** – confirming a TCP connection is actually establishing successfully (see the main Computer Networks guide's SYN/SYN-ACK/ACK diagram) before assuming a higher-layer problem.
- **Unexpected protocol or port activity** – traffic that shouldn't be there at all, useful for both troubleshooting and basic security triage.

Capture filters vs display filters. A **capture filter** (set before starting) limits what's actually recorded – useful for keeping a large, busy link's capture file manageable. A **display filter** (applied after capturing) just changes what's currently shown from an already-complete capture, without discarding anything. When in doubt, capture broadly and filter the display afterward – you can't display-filter data you never captured.

Worked Example: Diagnosing "The Website Is Slow"

A full walkthrough applying Section 2's OSI-layer method with the tools from Sections 3–5, for the classic vague complaint: "the website is slow."

1. **Layer 1–2 check:** confirm the reporting user's physical connectivity is fine – link lights up, no obvious cabling or Wi-Fi signal issues. Quickly ruled out; the problem isn't local physical connectivity.
2. **Layer 3 check (Section 3):** `ping` the website's IP directly. Replies come back consistently, with normal latency – basic routing and reachability are fine.
3. **DNS check (Section 4):** `dig` the domain against both the local resolver and a public one (`8.8.8.8`). Both resolve quickly to the same address – DNS isn't the bottleneck either.
4. **Layer 4 check:** confirm the relevant port (443) is actually reachable and accepting connections, not silently dropped somewhere. It connects fine.
5. **traceroute check (Section 3.2):** reveals a sharp, sustained latency jump starting at one specific hop and persisting through every hop after it – following the "sustained jump" pattern from Section 3.3, not an isolated timeout.
6. **Wireshark capture (Section 5):** a packet capture during a page load shows numerous TCP retransmissions – confirming actual packet loss, consistent with the latency jump found in the traceroute.

Root cause identified: packet loss and elevated latency at a specific point in the network path – not a DNS problem, not the web server itself, not the user's local connection. The methodical, layer-by-layer approach eliminated five other plausible causes with quick, cheap tests before finding the one that actually mattered, rather than guessing at the web server first just because that's where the slowness was *experienced*.

Symptom → Likely Cause Reference

A quick-reference starting point – not a substitute for the actual methodical process in Sections 2–6, but a reasonable first hypothesis to test when a familiar symptom appears.

SYMPTOM	LIKELY LAYER/CAUSE	FIRST TEST
No connectivity at all, any destination	L1/L2 – cable, switch port, Wi-Fi	Check link lights / Wi-Fi signal
Can ping IPs, but no websites load	DNS resolution failure	nslookup / dig (Section 4)
Some sites work, others don't	Possibly destination-specific, or DNS record issue	ping/dig the specific failing domain
Slow, inconsistent performance	Congestion or packet loss somewhere on the path	traceroute, watch for jumps (Section 3.3)
Connection resets mid-session	Firewall, misconfigured server, or app-layer issue	Wireshark capture, look for RST packets (Section 5.3)
Works on wired, fails on Wi-Fi only	L1/L2 – wireless-specific issue	Signal strength, interference, AP logs
Works locally, fails from outside the network	Port forwarding, firewall, or public DNS record	Check port forwarding rules, external DNS lookup

Use this table as a hypothesis, not a conclusion. These are common patterns, not guarantees – always verify with the actual layer-by-layer method (Section 2) rather than stopping at the first plausible-sounding match, especially for anything beyond a quick first guess.

Documentation & Escalation

8.1 WHY DOCUMENT EVEN AFTER IT'S FIXED

Once a problem is resolved, it's tempting to move on immediately – but a brief record of what happened, what was tested, and what actually fixed it pays off repeatedly: the same or a similar issue recurring becomes a five-minute fix instead of a from-scratch investigation, and a colleague facing something similar has a real reference instead of starting cold.

8.2 WHAT'S WORTH RECORDING

- **The original symptom**, as reported – in the reporter's own words, since that's often how it'll resurface next time.
- **What was tested, in order**, and the result of each test – this is effectively the layer-by-layer trail from Section 2.
- **The actual root cause**, once identified – not just "restarted the router and it worked," but what was actually wrong.
- **The fix applied**, and whether it's a permanent solution or a workaround that still needs a real fix later.

8.3 KNOWING WHEN TO ESCALATE

Not every problem is solvable at your level of access or expertise – and recognizing that early, rather than after hours of increasingly unproductive effort, is itself part of good troubleshooting method. Escalate when: the root cause appears to be outside your control (an upstream ISP, a third-party service), the fix requires access or authority you don't have, or the methodical process has genuinely exhausted the layers and tools available to you without a clear answer.

A good escalation includes your troubleshooting trail. Handing off "it's broken, please help" forces the next person to start from zero. Handing off "here's what I tested at each layer, here's what I found, here's where I got stuck" – essentially Section 8.2's documentation, written in real time – lets them pick up exactly where you left off instead of repeating your first six tests.

Quick Reference & Glossary

TERM	DEFINITION
ping	A tool sending ICMP Echo Requests to test basic reachability and measure latency.
tracert	A tool revealing the hop-by-hop path and latency to a destination.
TTL Exceeded	The ICMP message a router sends when a packet's TTL reaches zero, which tracert exploits.
nslookup / dig	Command-line tools for querying DNS and inspecting resolution results.
Wireshark	A packet capture and analysis tool for inspecting traffic at every layer.
Capture Filter	A filter limiting what traffic Wireshark actually records during capture.
Display Filter	A filter changing what's shown from an already-completed capture.
TCP Reset (RST)	An abrupt connection termination, often signaling a firewall block or application error.
Retransmission	The same TCP segment sent again, indicating suspected packet loss.
OSI-Layer Approach	Testing each network layer bottom-up to systematically isolate a fault's location.
Root Cause	The actual underlying reason a problem occurred, as distinct from just its symptom.

End of guide. Every tool in this guide answers a specific, narrow question – reachable? resolvable? what's actually in the traffic? Effective troubleshooting isn't about knowing every tool; it's about knowing which narrow question to ask next, in the right order, until the answer points at something you can actually fix.